



VirtualState Technical Whitepaper

Private, Host-Resistant Compute Continuity for Agentic AI

A technical pre-MVP architecture for turning heterogeneous servers into private, persistent, measurable and licensable compute environments for AI agents.

Pre-MVP technical draft

Open-source node vision

Public review version

Version 0.1 · 2026 · virtualstate.co

This document describes the intended technical architecture of VirtualState Node and related network services. It is not a security audit, a production specification or an investment document.

Abstract

Technical thesis

Agentic AI is shifting compute demand from ephemeral requests to persistent operational environments. A useful agent increasingly needs state, files, credentials, tools, policy constraints, snapshots and availability semantics. VirtualState proposes an infrastructure layer that turns heterogeneous host resources into private, persistent and measurable compute environments, exposed as compute licenses and backed by programmable contractual guarantees.

The initial technical target is not a full hardware-backed confidential computing system. Instead, VirtualState Node focuses on a pragmatic pre-MVP vertical slice: a persistent sandbox for the agent harness, encryption in transit and at rest, encrypted snapshots, remote attestation, a host-resistant Memory Vault for sensitive regions, external availability measurement and a smart-contract settlement path for downtime events.

CONTRIBUTIONS

Compute continuity

A runtime model for agents that need to persist, restore, and operate beyond a single human session.

Host-resistant privacy layer

A Memory Vault design for sensitive agent memory using split trust, ephemeral seeds, epoch rotation and dynamic layout changes.

Measurable guarantees

A contractual model where uptime, downtime and compensation can be expressed through licenses, oracles and settlement rules.

Open technical standard

A Node/Compute split: open-source node software where it builds trust, commercial trust layer where it creates guarantees.

Claim discipline

VirtualState does not claim software-only RAM-in-use encryption, absolute privacy, or equivalence to AMD SEV-SNP, Intel TDX or ARM CCA. It proposes a layered mitigation model for sensitive agent state.

Reading map and terminology

WHITEPAPER SECTIONS

1. Abstract and technical scope
2. Design goals and non-goals
3. System architecture overview
4. VirtualState Node components
5. Execution profiles
6. Confidential session lifecycle
7. Memory Vault and dynamic obfuscation
8. Attestation and trust tiers
9. Threat model
10. Compute licenses, oracles and contracts
11. Validation strategy and conclusion

CORE TERMS

Agent harness

The runtime around an AI system: memory, tools, credentials, tasks, connectors, state and policies. It may call a remote or local LLM.

Persistent sandbox

An isolated environment that preserves state, snapshots and operational continuity across sessions or restarts.

Compute license

A time-bound right to use defined compute resources under explicit availability, privacy and settlement conditions.

Host-resistant

A mitigation posture that raises the cost of host-side extraction of sensitive memory without claiming full protection from a malicious host.

Oracle

A measurement component that converts availability and operational events into verifiable contract inputs.

Scope note

This whitepaper intentionally avoids production-grade constants, endpoint schemas, hardening scripts, vault layout parameters and key-management details. The purpose is to attract technical review and collaboration without publishing an exploitable implementation manual before the first audit cycle.

Problem statement: persistent agents need environments

From reactive calls to operational runtimes

A reactive chatbot consumes resources during a request. A persistent agent consumes continuity: state, tools, credentials, scheduled work, snapshots, and recovery semantics. This changes the infrastructure problem from “where do we run inference?” to “where does the agent live operationally?”

Dimension	Reactive chatbot	Persistent agent
Execution	Prompt-response spikes	Long-running tasks and parallel processes
State	Session history or limited context	Memory, files, credentials, configuration, snapshots
Privacy	Data sent to API/platform	Need for runtime perimeter, vault, and context control
Availability	Conventional SLA	Continuity, restoration, and programmable consequences
Economics	Subscription or usage bill	Compute license with explicit terms and settlement

VIRTUALSTATE TARGET

Persistence

Keep agent state and snapshots across operational windows.

Practical privacy

Reduce exposure of sensitive regions to the host and operator.

Measured availability

Convert heartbeat and downtime into auditable events.

Contractual semantics

Encode resource rights, thresholds and remedies as compute licenses.

Design intuition: the bottleneck is not only raw compute. It is the right to exist as a private, restorable and contractually meaningful runtime.

Design goals and explicit non-goals

DESIGN GOALS

- Host heterogeneous compute through an auditable open-source node.
- Run a persistent sandbox that contains the agent harness, not necessarily the LLM.
- Encrypt traffic, persistent storage and snapshots by default.
- Protect sensitive agent material through a Memory Vault rather than attempting whole-VM privacy in the MVP.
- Condition session material on attestation, policy and client participation.
- Measure heartbeat and availability as first-class protocol events.
- Represent capacity as compute licenses with clear technical and settlement terms.

NON-GOALS

- No claim of software-only confidential computing or full RAM-in-use encryption.
- No full custom production hypervisor from day one.
- No open permissionless marketplace in the initial technical slice.
- No guarantee against malicious firmware, BMC implants, invasive physical attacks or modified hardware in the MVP.
- No tokenized investment product or ownership claim over hardware not controlled or certified.
- No assumption that blockchain alone solves trust in execution or confidentiality.

Security posture in one line

VirtualState increases the cost and reduces the window of host-side extraction of sensitive agent state; it does not make a hostile host cryptographically impossible.

Engineering discipline

The architecture must be explainable as a sequence of verifiable artifacts: node, sandbox, guest agent, vault, attestation, heartbeat, license and settlement. If a component cannot be observed, tested or audited, it does not belong in the MVP claim surface.

System architecture overview

Applications

Private AI · persistent agents · runtimes · open-source models

VirtualState Compute

Marketplace · API · reputation · support · certified providers

Contracts + oracles

Licenses · uptime · downtime · collateral · compensation · signed logs

VirtualState Node

Lightweight runtime · sandbox manager · policy engine · metrics

Private sandbox

Isolation · state · snapshots · encrypted storage · Memory Vault

Host infrastructure

Cloud · bare metal · own servers · verified third parties

CONTROL PLANE VS DATA PLANE

Control plane

- Policy selection
- License lifecycle
- Attestation tokens
- Oracle reporting
- Settlement triggers

Data plane

- Sandbox execution
- Encrypted I/O
- Agent state
- Memory Vault access
- Snapshots and restore

Boundary principle

The platform should never confuse contractual integrity with execution confidentiality. Contracts can settle availability; attestation and runtime controls mitigate execution risk.

VirtualState Node: technical component map



Execution profiles

VirtualState Node should support more than one execution profile over time. The MVP focuses on a general Linux sandbox because it covers realistic agent harnesses; Wasm/WASI is a strong profile for specialist micro-agents; hardware confidential computing is treated as a premium future tier.

Profile	Best fit	MVP status	Security implication
General Linux sandbox	Agent harnesses that need CLI tools, local files, Python scripts, databases, connectors and broad OS freedom.	Primary MVP path	Harder to protect completely; Memory Vault narrows the sensitive surface.
Wasm/WASI micro-agent	Single-purpose agents, API workers, validators, file processors, deterministic functions.	Secondary profile / later	Smaller surface, capability-based access, simpler attestation, better portability.
Hardware confidential node	High-value workloads on AMD SEV-SNP, Intel TDX, ARM CCA or equivalent.	Future premium tier	Closest to real memory-in-use protection; limits hardware availability and decentralization.

WHY NOT FORCE ONE RUNTIME?

Different agents have different operational physics

A general agent may need a full Linux environment. A specialist agent may be safer and cheaper as a Wasm module. A highly regulated workload may require a hardware TEE. VirtualState should expose privacy and trust as explicit levels, not as a single binary claim.

DESIGN DECISION

The MVP should validate the hardest narrative point first: a persistent Linux agent harness with practical host-resistance and contractual availability semantics.

Confidential session lifecycle

1. Publish capacity

Node advertises resources and a candidate guarantee level.

2. Launch sandbox

A minimal sandbox boots and generates an ephemeral session keypair.

3. Request attestation

Node submits `node_id`, `license_id`, runtime hash, policy, measurements and sandbox public key.

4. Validate policy

Attestation service verifies software measurements and TPM quote when available.

5. Release VS share

VirtualState sends an encrypted ephemeral share to the sandbox.

6. Client share

Client validates attestation and sends an E2EE epoch share.

7. Derive epoch seed

Sandbox derives `seed_epoch` and configures Memory Vault layout and rotation.

8. Rotate / revoke

Epochs destroy old material; abnormal reboot, heartbeat loss or policy mismatch stops future shares.

```
abstract seed derivation (non-production)
seed_epoch = KDF(
    vs_share || client_share || vm_nonce || license_id || node_id || epoch
)
policy_epoch = Policy(runtime_hash, attestation_level, license_terms, epoch)
if verify(policy_epoch) and heartbeat_ok:
    MemoryVault.reconfigure(seed_epoch, policy_epoch)
else:
    revoke_future_shares(license_id, node_id)
```

Important

This is a conceptual state machine, not a production key schedule. Production parameters, KDF context separation and failure handling must be finalized under security review.

Memory Vault: narrowing the sensitive surface

Design purpose

The MVP should not try to hide every byte of the guest OS. It should protect the material that matters most: API keys, delegated wallets, sensitive context, internal instructions, task queues and snapshot-bound secrets.

Protected element	MVP treatment
Credentials and API keys	Never passed as persistent environment variables; stored and retrieved through the vault.
Delegated wallet / budget	Encrypted key material, capability-gated operations and permission rotation.
Sensitive context	Fragmented and encrypted before snapshot or disk persistence.
Internal instructions / task queue	Non-linear structures, chaff and epoch-based layout rotation.
Snapshots	Encrypted before leaving the sandbox; hashed and signed for integrity.
Logs	Signed operational events without secret-bearing content.

OPERATIONAL HARDENING

- mlock or equivalent where possible
- avoid swap for sensitive processes
- disable core dumps
- zeroize aggressively
- short-lived buffers
- least-privilege process separation
- dump-comparison tests

```
conceptual vault API
vault.put(label, plaintext, policy)
  -> fragment(policy)
  -> encrypt(epoch_key)
  -> place(layout_map)
  -> zeroize(plaintext)

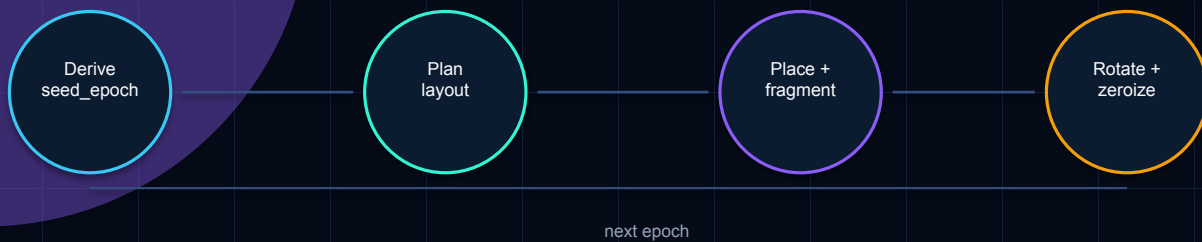
vault.get(label, capability)
  -> verify(capability)
  -> reconstruct_in_short_lived_buffer()
  -> use_once_or_rewrap()
```

Security claim

A casual or delayed RAM dump should not reveal stable, obvious secret structures. A live, instrumented malicious host remains a residual risk.

Dynamic obfuscation and epoch rotation

Dynamic obfuscation is used as a mitigation layer for sensitive regions, not as a cryptographic replacement for hardware memory encryption. The goal is to reduce stable structure, reduce useful dwell time and make offline extraction materially less useful.



```
epoch transition (conceptual)
for each epoch:
    seed_epoch      = derive_epoch_material(...)
    layout_map      = deterministic_plan(seed_epoch, vault_policy)
    chaff_profile    = policy_bound_noise(seed_epoch, risk_level)
    rewrap(secret_fragments, previous_epoch_key, epoch_key)
    relocate(secret_fragments, layout_map)
    zeroize(previous_epoch_material)
    emit_signed_event(epoch_id, policy_hash, non_secret_metrics)
```

What it improves

- Secret locality
- Static signatures
- Snapshot leakage
- Delayed/offline memory analysis

What it does not solve

- Fully malicious live host
- Compromised firmware
- Side channels not yet modeled
- Hardware-level invasive attacks

Remote attestation and TPM 2.0

Role of TPM 2.0

TPM 2.0 does not encrypt memory in use. Its value is to provide a hardware-rooted identity and measurement channel that can condition the release of ephemeral session material on a measured and approved boot/runtime state.

Mechanism	Provides	Limitation
TPM 2.0	Quotes, PCRs and platform identity.	Does not protect RAM from an authorized host during execution.
UEFI Secure Boot	Reduces unauthorized bootloader/kernel paths.	Depends on correct configuration and key chain.
Measured Boot	Records boot measurements for remote verification.	Measures what booted, not everything that happens later.
Runtime attestation	Can measure binaries and loaded components.	Adds complexity and may create false positives.
Hardware TEE	Stronger memory-in-use protection against host/hypervisor.	Not required for MVP due to limited hardware reach.

ATTESTATION TOKEN - CONCEPTUAL FIELDS

```
attestation_token = {
  node_id, license_id, attestation_level,
  runtime_hash, policy_hash, sandbox_pubkey,
  measurement_set: {pcrs?, secure_boot?, measured_boot?, runtime_components?},
  issued_at, expires_at, revocation_endpoint,
  signature: sign_attestation_service(token_body)
}
```

Policy rule

No attestation, no strong guarantees. Open nodes may run freely; VirtualState Compute guarantees require policy-backed measurement and revocation paths.

Trust levels and privacy promises

The network should not pretend that every node offers the same confidentiality level. Trust is exposed as a tiered, auditable property that balances decentralization, hardware availability and defensible claims.

0

Open Node

No trust requirements

Free use; no commercial confidentiality or collateral guarantees.

1

Software-Attested

Signed runtime, hashes, E2EE, logs

Limited trust for demos, development and low-risk workloads.

2

TPM-Verified

TPM 2.0, Secure Boot, measured boot

Initial attestation and conditional release of ephemeral material.

3

Certified Provider

TPM + maintenance policy + reputation + collateral

Marketplace-ready node with economic guarantees and trust scoring.

4

Hardware Confidential

SEV-SNP, TDX, CCA or equivalent

Premium future tier with real confidential computing properties.

Product implication

TPM 2.0 should not be required to run VirtualState Node, but it should become the minimum recommended trust root for strong guarantees inside VirtualState Compute.

Threat model: assets, adversaries and assumptions

ASSETS PROTECTED

- Agent credentials and API keys
- Delegated wallet or budget keys
- Sensitive context and private task queues
- Encrypted snapshots and restore integrity
- License state and settlement conditions
- Availability telemetry and signed logs
- Node reputation and trust tier claims

ADVERSARIES CONSIDERED

- External network attacker
- Provider reading disk or snapshots
- Host performing occasional RAM dumps
- Active software host with instrumentation
- Compromised firmware/BMC/BIOS
- Advanced physical or hardware-modified attacker
- Malicious or buggy oracle participant

ASSUMPTIONS

- Cryptographic primitives are correctly implemented and audited before production use.
- Clients can verify attestation tokens before releasing their share of session material.
- VirtualState Compute may refuse strong guarantees for nodes that fail measurement policy.
- Open-source code transparency is not treated as a vulnerability; secrets and policy material are not embedded in the codebase.
- Availability can be measured more objectively than confidentiality; therefore oracles are stronger for downtime than for proving absence of host access.

Core threat model statement

The MVP aims to resist network attackers and disk/snapshot exposure, mitigate casual or delayed host memory inspection, and explicitly not guarantee protection against a fully active malicious host or compromised hardware without stronger trust tiers.

Threat mitigation matrix

Adversary / event	MVP mitigation	Acceptable claim
External network attacker	E2EE, authenticated channels, pinned policy where applicable.	Traffic content and session material are protected in transit.
Disk or snapshot reader	Encrypted snapshots, non-persistent host keys, hash/signature integrity.	Snapshots should not expose secrets if configuration is correct.
Occasional RAM dump	Memory Vault, non-stable layout, fragmentation, chaff, zeroization.	Sensitive memory should not appear as stable, trivial structures.
Active host instrumentation	Reduced exposure window and dynamic layout; no full guarantee.	Raises cost; does not eliminate risk.
Firmware/BMC/BIOS manipulation	Measured boot and TPM policy in verified/certified tiers.	Requires certified nodes; not solved by base open node.
Downtime / silent failure	Heartbeat, external checks, signed events and threshold-based settlement.	Availability can trigger compensation or revocation.
Oracle disagreement	Quorum, trimmed mean or dispute state.	Settlement can pause rather than blindly execute.

PROHIBITED CLAIMS

- Full RAM-in-use encryption without hardware
- Absolute privacy against the host
- TPM as a substitute for SEV-SNP/TDX/CCA
- Perfect confidentiality in an open distributed network from MVP
- Oracles proving that memory was never inspected
- Tokenized compute as a guaranteed investment product

ALLOWED CLAIMS

- Host-resistant layer for sensitive memory
- Remote attestation and conditional release of ephemeral material
- TPM 2.0 as a minimum trust root for verified nodes
- Split trust and ephemeral seeds to reduce attack surface
- Hardware confidential computing as future premium tier
- Contractual availability semantics through oracles and licenses

Compute licenses: technical resource rights

Definition

A compute license is a programmable right to use defined compute resources under explicit technical and availability conditions. It is not hardware ownership and should not be positioned as an investment claim.

Field	Example	Technical role
resource_vector	8 vCPU · 32 GB RAM · 500 GB encrypted storage	Defines the capacity envelope.
duration	30 days / renewable / block hours	Defines validity and accounting window.
trust_level	Open, Software-Attested, TPM-Verified, Certified	Defines privacy and guarantee claim surface.
uptime_target	98.5% / 99.5% / policy-specific	Defines expected availability.
downtime_threshold	2h tolerated or % per period	Defines when settlement may trigger.
collateral_policy	deposit, credit, auto-buyback or extension	Defines remedy semantics.
oracle_policy	heartbeat, external probes, quorum/dispute	Defines evidence source.

LIFECYCLE



Design principle

Collateral does not remove technical risk. It converts part of availability risk into a programmable obligation.

Smart contracts and oracle settlement

The blockchain layer is used as contractual integrity infrastructure: registering license terms, collateral, oracle events and deterministic settlement. It should not be the main confidentiality mechanism.

```
// Solidity-like sketch. Not production code.
struct ComputeLicense {
    bytes32 id;
    address client;
    address provider;
    ResourceVector resources;
    TrustLevel trustLevel;
    uint64 startTime;
    uint64 endTime;
    uint64 downtimeThreshold;
    uint256 collateralLocked;
    SettlementPolicy policy;
    LicenseState state;
}

function reportAvailability(bytes32 id, OracleEvent event, bytes sig) external onlyOracle {
    require(verifyOracle(sig, event));
    availabilityLog[id].push(event);
    if (downtimeExceeded(id)) { state[id] = LicenseState.SettlementPending; }
}

function settle(bytes32 id) external {
    require(state[id] == LicenseState.SettlementPending);
    applyPolicy(id, availabilityLog[id], license[id].policy);
}
```

ORACLE DESIGN

- Multiple measurement points should be supported over time: node heartbeat, external probes, client-side verification and service-level events.
- VirtualState may operate an initial oracle, but production guarantees should not depend on VirtualState being the only source of truth.
- Disagreement should pause settlement or trigger dispute rather than create false automatic penalties.
- Availability evidence should be signed and content-free: enough to settle conditions, not enough to leak customer data.

Web3 boundary

VirtualState should be read as infrastructure using smart contracts, not as a token-first protocol. Compute licenses describe usage rights and service remedies, not yield or passive return.



Availability telemetry and event semantics

Event	Detection path	Contractual response
Heartbeat loss	External health checks + signed guest/node heartbeat.	Suspend session, revoke future shares, mark downtime.
Unauthorized restart	Changed nonce, counter, log sequence or measurement.	Invalidate session and require re-attestation.
Runtime mismatch	Hash/version does not match approved policy.	Deny ephemeral material.
Downtime threshold exceeded	Windowed calculation over signed events.	Trigger credit, extension, compensation or buyback.
Declared maintenance	Registered before license issuance or within policy.	No penalty or different penalty according to SLA.
Oracle discrepancy	Quorum, trimmed mean, dispute or manual review.	Pause settlement until evidence converges.

SIGNED EVENT FORMAT - CONCEPTUAL

```
availability_event = {  
  license_id, node_id, sandbox_id,  
  event_type: HEARTBEAT | DOWNTIME | RESTART | POLICY_MISMATCH,  
  observed_at, window_id, sequence_no,  
  non_secret_metrics: {latency?, status?, uptime_counter?},  
  previous_event_hash,  
  signature: sign_oracle_or_guest(event_body)  
}
```

Telemetry privacy rule

Telemetry should be useful for settlement and reputation without becoming a side channel for customer content, secret names, internal task descriptions or business-sensitive metadata.

Open-source security and value capture

VirtualState Node

- Open-source node software
- Sandbox creation and resource publication
- Availability measurement primitives
- Guest agent and vault interfaces
- Auditable technical base
- Usable outside commercial network

VirtualState Compute

- Commercial trust layer
- Marketplace and API
- Attestation service and policy registry
- Provider certification and reputation
- Oracle coordination
- Support and settlement infrastructure

Security must survive transparency

The node being open source should improve trust rather than weaken security. Sensitive guarantees depend on policy, attestation, ephemeral material, client participation, measurement and certification—not on hiding code.

WHY COMPUTE STILL MATTERS

- Standardization: open node implementations make the ecosystem legible and auditable.
- Trust operations: certification, reputation and support are operational services, not just code artifacts.
- Liquidity: a marketplace becomes useful only when supply, demand, quality signals and settlement are coordinated.
- Policy enforcement: strong guarantees require attestation policy, revocation, dispute handling and support loops.
- Commercial integration: enterprise customers need onboarding, compliance boundaries and reliability commitments.

Open source does not destroy the model; it standardizes the technical substrate.



Validation strategy and technical invitation

END-TO-END MVP VALIDATION

- Publish host resources through VirtualState Node.
- Create a compute license for a persistent Linux sandbox.
- Run a simple agent harness that preserves state across sessions.
- Execute attestation and split-trust session material release.
- Protect a secret through Memory Vault and epoch rotation.
- Show a comparative memory dump without and with the privacy layer.
- Measure uptime with signed heartbeat events.
- Simulate downtime beyond threshold.
- Trigger compensation or buyback in testnet or deterministic mock.
- Display signed logs that explain what happened without leaking content.

TECHNICAL REVIEW AREAS

- Virtualization and sandbox hardening
- TPM / measured boot / attestation policy
- Key management and split-trust lifecycle
- Memory safety and zeroization behavior
- Dump-comparison methodology
- Oracle robustness and false-positive handling
- Smart contract settlement semantics
- Provider certification and operational security

Conclusion

VirtualState is technically interesting because it connects three problems that are usually treated separately: persistent agent execution, host-resistant handling of sensitive state, and programmable availability guarantees. The first milestone is not a full decentralized cloud. It is a credible vertical slice that can be inspected, attacked, improved and iterated.

Public technical draft

This document is intended to attract scrutiny from security engineers, virtualization specialists, distributed-systems builders, smart-contract developers and agent infrastructure researchers.

virtualstate.co